PONTIFÍCIA UNIVERSIDADE CATÓLICA DO RIO DE JANEIRO

Alocação preditiva de recursos em nuvem via previsão de carga com Redes Neurais para minimização de custos operacionais

Gabriel Lobo de Sant'Anna Rosas

PROJETO FINAL DE GRADUAÇÃO

CENTRO TÉCNICO CIENTÍFICO - CTC

DEPARTAMENTO DE INFORMÁTICA

Curso de Graduação em Engenharia da Computação

Rio de Janeiro, Novembro de 2026

Pontifícia Universidade Católica
do Rio de Janeiro



Gabriel Lobo de Sant'Anna Rosas

Alocação preditiva de recursos em nuvem via previsão de carga com Redes Neurais para minimização de custos operacionais

Relatório de Projeto Final, apresentado ao programa de Graduação do Departamento de Informática da PUC-Rio como requisito parcial para a obtenção do título de Engenheiro de Computação.

Orientador: Adriano Francisco Branco
Co-orientador: Jonatas dos Santos Grosman

Rio de Janeiro
Novembro de 2026

Resumo

Lobo de Sant'Anna Rosas, Gabriel. Francisco Branco, Adriano. dos Santos Grosman, Jonatas. **Alocação preditiva de recursos em nuvem via previsão de carga com Redes Neurais para minimização de custos operacionais**. Rio de Janeiro, 2026. 28 p. Relatório de Projeto Final – Departamento de Informática. Pontifícia Universidade Católica do Rio de Janeiro.

Provedores de nuvem precisam decidir continuamente quantas instâncias manter ativas: recursos em excesso elevam o custo, e recursos insuficientes violam o acordo de nível de serviço. A alocação reativa só ajusta a capacidade depois de observar a mudança na demanda e, por isso, chega atrasada nos picos. Este trabalho investiga o quanto a qualidade da previsão de carga reduz o custo de alocação sem aumentar as violações de serviço. Para atribuir as diferenças ao previsor, a política de alocação é mantida fixa e apenas o previsor muda. São comparados métodos tradicionais de previsão (ingênuo, média móvel, sazonal, Holt-Winters e ARIMA) e uma rede neural recorrente do tipo GRU, além de uma política reativa e de um dimensionamento estático no pico, sobre séries de utilização agregadas a partir do rastro público de um aglomerado da Alibaba e avaliadas em um centro de dados simulado. As métricas são o custo normalizado pelo de um oráculo e a fração de intervalos com violação. Os resultados preliminares mostram que a previsão reduz muito o custo frente ao dimensionamento estático e de forma modesta frente à política reativa, e que a GRU é competitiva com o ARIMA, sem vantagem clara. Esta etapa também define o ambiente e o plano do Projeto Final II.

Palavras-chave

Computação em nuvem; Alocação de recursos; Previsão de carga; Redes neurais; Acordo de nível de serviço.

Abstract

Lobo de Sant'Anna Rosas, Gabriel. Francisco Branco, Adriano (Advisor). dos Santos Grosman, Jonatas (Co-advisor). **Predictive cloud resource allocation via workload forecasting with neural networks for operating cost minimization**. Rio de Janeiro, 2026. 28 p. Undergraduate Thesis – Departamento de Informática. Pontifícia Universidade Católica do Rio de Janeiro.

Cloud providers must continuously decide how many instances to keep active: excess resources raise costs, while insufficient resources violate the service level agreement. Reactive allocation adjusts capacity only after observing a change in demand and therefore lags behind peaks. This work investigates how much the quality of workload forecasting reduces allocation cost without increasing service violations. To attribute differences to the forecaster, the allocation policy is kept fixed and only the forecaster changes. Traditional forecasting methods (naive, moving average, seasonal, Holt-Winters and ARIMA) and a recurrent neural network of the GRU type are compared, together with a reactive policy and a static peak sizing, using utilization series aggregated from a public Alibaba cluster trace and evaluated in a simulated data center. The metrics are the cost normalized by that of an oracle and the fraction of intervals with violations. Preliminary results show that forecasting greatly reduces cost compared with static sizing and only modestly compared with the reactive policy, and that the GRU is competitive with ARIMA, with no clear advantage. This stage also defines the environment and the plan for the second part of the project.

Keywords

Cloud computing; Resource allocation; Workload forecasting; Neural networks; Service level agreement.

Sumário

1 Introdução **7**
1.1 Motivação 7
1.2 Definição do problema 7
1.3 Justificativa 8
1.4 Escopo e delimitação 9
1.5 Organização do documento 9

2 Situação Atual **10**
2.1 Conceitos e terminologia 10
2.2 Abordagens para previsão de carga 13
2.3 Abordagens para alocação de recursos 16
2.4 Comparativo dos trabalhos relacionados 18
2.5 Tecnologias utilizadas 20

3 Objetivos do trabalho **23**

4 Plano de ação **24**

5 Cronogramas **25**

Referências **26**

Lista de tabelas

Tabela 2.1 Comparativo entre os trabalhos relacionados e a proposta deste trabalho. 19

1
Introdução

1.1
Motivação

A computação em nuvem consolidou-se como a principal forma de disponibilizar infraestrutura computacional para empresas e instituições. Nesse modelo, o provedor mantém grandes centros de dados e precisa decidir, de forma contínua, quanto de processamento, memória e armazenamento deve estar disponível para atender à demanda de seus clientes. Como a demanda tende a crescer, impulsionada sobretudo por aplicações de Inteligência Artificial e de análise de grandes volumes de dados, a gestão eficiente desses recursos tornou-se um fator central para a sustentabilidade econômica dos provedores (ROSSI et al., 2025).

A dificuldade está no fato de que a carga de trabalho não é constante. Ela apresenta picos, padrões periódicos e variações aparentemente aleatórias. Diante disso, a abordagem mais comum é a alocação reativa, na qual o sistema ajusta os recursos apenas depois de observar uma mudança na demanda, normalmente por meio de regras de limiar sobre a utilização. Como a reação ocorre com atraso, esse tipo de estratégia tende a degradar o desempenho durante os picos e a desperdiçar recursos nos períodos de baixa utilização (BODRA; KHAIRNAR, 2025).

Esse cenário cria um dilema permanente. Manter recursos em excesso (superprovisionamento) garante a qualidade do serviço, mas eleva o custo operacional. Manter recursos de menos (subprovisionamento) reduz o custo, mas provoca violações dos Acordos de Nível de Serviço (*Service Level Agreements*, SLA) firmados com os clientes. Uma alternativa é a alocação preditiva, que utiliza o histórico de uso para antecipar a demanda futura e provisionar recursos antes que eles sejam necessários (BODRA; KHAIRNAR, 2025). Nesse contexto, prever a carga com precisão passa a ser uma pré-condição para otimizar a alocação (ROSSI et al., 2025).

A previsão de carga em nuvem foi tradicionalmente tratada com modelos estatísticos lineares, como o ARIMA (CALHEIROS et al., 2015). Mais recentemente, redes neurais profundas, em especial as arquiteturas recorrentes, passaram a ser aplicadas ao problema com bons resultados, inclusive em abordagens multivariadas que consideram simultaneamente o uso de processador e de memória (XU et al., 2022).

1.2
Definição do problema

O problema tratado neste trabalho tem duas partes encadeadas. A primeira é um problema de previsão: dado o histórico de utilização de recursos de um *cluster*, amostrado em intervalos regulares, estimar a demanda de processador e

de memória em um horizonte curto. A segunda é um problema de decisão: a partir dessa estimativa, determinar quantas instâncias de recursos (máquinas virtuais ou contêineres) devem permanecer ativas em cada intervalo.

De forma preliminar, a decisão pode ser formulada como a minimização do custo operacional sujeita a uma restrição de qualidade de serviço:

$$\min_{n_1,\ldots,n_T} \sum_{t=1}^{T} c \cdot n_t \quad \text{sujeito a} \quad \frac{1}{T}\sum_{t=1}^{T} \mathbf{1}\left[\, d_t > k \cdot n_t \,\right] \leq \varepsilon, \tag{1-1}$$

em que n_t é o número de instâncias ativas no intervalo t, c é o custo de uma instância por intervalo, k é a capacidade de uma instância, d_t é a demanda real observada, $\mathbf{1}[\cdot]$ é a função indicadora e ε é a fração máxima de intervalos com violação de SLA tolerada. Como d_t não é conhecida no momento da decisão, a escolha de n_t depende da previsão $\hat{d}_t$, e a qualidade dessa previsão afeta diretamente o custo e o número de violações. Essa formulação será refinada no Capítulo 3.

1.3 Justificativa

A relevância do problema é econômica e operacional. Cada ponto percentual de recursos ociosos representa custo para o provedor, e cada violação de SLA representa perda de qualidade percebida pelo cliente e, eventualmente, penalidades contratuais.

Apesar do volume de trabalhos sobre previsão de carga, a maior parte deles avalia os modelos apenas por métricas de erro pontual, como RMSE e MAPE. Rossi et al. (2025) mostram que essas métricas, isoladamente, não oferecem uma visão clara da utilidade dos modelos para a gestão de recursos, e apontam que integrar os modelos de previsão aos mecanismos de alocação em um mesmo fluxo ainda é uma questão em aberto. O esDNN (XU et al., 2022) chega a aplicar suas previsões em uma simulação de escalonamento automático de máquinas, mas compara o resultado apenas com uma estratégia baseada na média das observações anteriores, sem confrontá-lo com outros métodos de previsão sob a mesma política de alocação.

Do lado da alocação, a revisão de Bodra e Khairnar (2025) observa que a literatura carece de comparações quantitativas sistemáticas entre abordagens e que técnicas de aprendizado por reforço profundo, embora promissoras, são sensíveis ao projeto da função de recompensa e aos hiperparâmetros, o que dificulta sua adoção confiável. Isso favorece abordagens em que a previsão e a decisão sejam componentes separados, mais fáceis de interpretar e de avaliar.

Dessa forma, há espaço para um trabalho que avalie de ponta a ponta o efeito da previsão sobre a alocação, medindo custo e violações de SLA, e que compare redes neurais e métodos tradicionais sob as mesmas condições, utilizando dados públicos de *clusters* reais.

1.4 Escopo e delimitação

Para que o trabalho seja viável no tempo disponível, o escopo é delimitado da seguinte forma.

Incluído no escopo. O trabalho utilizará rastros públicos de utilização de *clusters*, especificamente o Alibaba Cluster Trace (Alibaba Group, 2018) e o Google Cluster Trace (WILKES, 2020), agregados em séries temporais de uso de processador e de memória. Serão considerados horizontes de previsão curtos, compatíveis com decisões de alocação na escala de minutos. A alocação será avaliada em um centro de dados simulado, com um modelo de custo simplificado baseado no custo por instância ativa e uma métrica de violação de SLA baseada na demanda não atendida. A abordagem proposta, baseada em rede neural, será comparada com uma política reativa por limiar e com métodos tradicionais de previsão.

Fora do escopo. Não fazem parte deste trabalho a implantação em uma infraestrutura de nuvem real, o posicionamento detalhado de máquinas virtuais em servidores físicos e a migração entre eles, a otimização de consumo de energia, cenários com múltiplos provedores e o uso de aprendizado por reforço para a tomada de decisão.

1.5 Organização do documento

O restante deste documento está organizado da seguinte forma. O Capítulo 2 apresenta a terminologia básica e as principais abordagens existentes para previsão de carga e alocação de recursos em nuvem, com suas limitações, além das tecnologias que serão utilizadas. O Capítulo 3 descreve a solução proposta, seus objetivos específicos e o que será efetivamente implementado. O Capítulo 4 revisa o plano de ação, comparando as atividades propostas com as realizadas. Por fim, o Capítulo 5 apresenta os cronogramas do Projeto Final I e do Projeto Final II.

2
Situação Atual

Este capítulo apresenta o contexto necessário para compreender o problema tratado neste trabalho. A Seção 2.1 define os conceitos e a terminologia básica. As Seções 2.2 e 2.3 descrevem, respectivamente, as abordagens existentes para previsão de carga e para alocação de recursos em nuvem, destacando suas limitações. A Seção 2.4 compara os trabalhos mais próximos desta proposta, e a Seção 2.5 apresenta as tecnologias que serão utilizadas.

2.1
Conceitos e terminologia

2.1.1
Elasticidade, carga de trabalho e séries temporais

Uma das características essenciais da computação em nuvem é a elasticidade rápida, isto é, a capacidade de provisionar e liberar recursos rapidamente, em alguns casos de forma automática, acompanhando a demanda (MELL; GRANCE, 2011). Herbst, Kounev e Reussner (2013) definem elasticidade como o grau em que um sistema se adapta a mudanças na carga de trabalho, provisionando e liberando recursos de forma autônoma, de modo que, a cada instante, os recursos disponíveis correspondam o mais fielmente possível à demanda corrente. Essa definição deixa claro que a elasticidade tem duas falhas possíveis: oferecer recursos de mais ou recursos de menos.

A carga de trabalho (*workload*) de um *cluster* é o conjunto de demandas submetidas a ele, como tarefas em lote, serviços de longa duração e requisições de usuários. Do ponto de vista da infraestrutura, essa carga se manifesta como utilização de recursos, principalmente processador e memória. Quando essa utilização é agregada em intervalos regulares de duração Δ, obtém-se uma série temporal $y_1, y_2, \ldots, y_t$, em que y_t é a utilização média (ou máxima) no intervalo t. Séries de carga em nuvem costumam combinar tendência, padrões periódicos (por exemplo, diários e semanais), ruído e picos abruptos, o que torna sua previsão difícil (XU et al., 2022).

Prever a carga significa estimar $\hat{y}_{t+h}$, o valor da série h intervalos à frente, a partir de uma janela com as w observações mais recentes. O parâmetro h é o horizonte de previsão e w é o tamanho da janela de entrada. A previsão é univariada quando usa apenas a própria série e multivariada quando incorpora outras séries correlacionadas, como prever o uso de processador a partir do histórico de processador e de memória (XU et al., 2022).

Pesquisas sobre o tema dependem de rastros (*traces*) públicos de *clusters* reais, que registram eventos e uso de recursos de máquinas, tarefas e contêineres. O rastro mais recente do Google cobre oito células do gerenciador Borg durante o

mês de maio de 2019 (WILKES, 2020). O rastro de 2018 da Alibaba cobre cerca de 4.000 máquinas de um *cluster* de produção ao longo de oito dias, com serviços *online* e tarefas em lote compartilhando as mesmas máquinas, e inclui tabelas de uso de recursos por máquina e por contêiner (Alibaba Group, 2018). Como esses rastros registram eventos e medições em granularidade fina, transformá-los em séries temporais regulares é uma etapa de pré-processamento necessária, que faz parte deste trabalho.

A granularidade dessa agregação varia entre os trabalhos, mas intervalos de cinco minutos são os mais usados com esses rastros. Rossi et al. (2025) agregam os rastros do Google e da Alibaba em intervalos de cinco minutos e preveem a demanda dez minutos à frente, horizonte que consideram suficiente para a maioria das aplicações de alocação. O esDNN também adota cinco minutos como intervalo de previsão e de decisão na simulação de escalonamento (XU et al., 2022). Trabalhos com outros tipos de dado usam escalas diferentes, como uma hora para requisições a servidores *web* (CALHEIROS et al., 2015) ou um minuto para dados de produção de um provedor (PAN et al., 2023). O ponto comum é que o intervalo deve ser longo o bastante para que uma nova instância possa ser iniciada dentro dele (CALHEIROS et al., 2015).

2.1.2 Acordos de nível de serviço

A qualidade de um serviço em nuvem é expressa por três conceitos relacionados (BEYER et al., 2016). Um indicador de nível de serviço (*Service Level Indicator*, SLI) é uma medida quantitativa de algum aspecto do serviço, como latência, disponibilidade ou fração de requisições atendidas com sucesso. Um objetivo de nível de serviço (*Service Level Objective*, SLO) é um valor-alvo ou uma faixa aceitável para um SLI, por exemplo, atender 99% das requisições dentro de um limite de tempo. Por fim, um acordo de nível de serviço (*Service Level Agreement*, SLA) é o contrato entre provedor e cliente que formaliza esses objetivos e define as consequências, geralmente financeiras, de não cumpri-los.

Na literatura de alocação de recursos, a violação de SLA é frequentemente aproximada por medidas derivadas da utilização, já que os rastros públicos não registram a latência percebida pelos usuários. Beloglazov e Buyya (2012), por exemplo, medem a fração do tempo em que os servidores ativos operam com utilização máxima, situação em que o desempenho das aplicações é presumivelmente degradado. Rossi et al. (2025) consideram atendida a demanda que fica abaixo da capacidade prevista.

Este trabalho adota uma aproximação do mesmo tipo. O SLI é a fração da demanda atendida pela capacidade provisionada, e ocorre uma violação no intervalo t quando a demanda real supera a capacidade disponível, isto é, quando $d_t > k \cdot n_t$, na notação da Equação 1-1. O parâmetro ε daquela equação faz o papel do SLO. Essa simplificação não captura efeitos como filas ou degradação

gradual de desempenho perto da saturação, mas permite comparar estratégias de alocação de forma consistente e reproduzível.

2.1.3 Provisionamento reativo e preditivo

O escalonamento automático (*autoscaling*) é o mecanismo que ajusta os recursos alocados a uma aplicação sem intervenção humana. Ele pode ser horizontal, quando altera o número de instâncias (máquinas virtuais ou contêineres), ou vertical, quando altera a capacidade de cada instância. Este trabalho trata apenas do escalonamento horizontal.

Lorido-Botran, Miguel-Alonso e Lozano (2014) classificam as técnicas de escalonamento automático em cinco famílias: regras baseadas em limiares, aprendizado por reforço, teoria de filas, teoria de controle e análise de séries temporais. Do ponto de vista deste trabalho, a distinção mais importante é entre abordagens reativas e preditivas. No provisionamento reativo, a decisão no instante t é tomada a partir do que já foi observado, normalmente a utilização corrente. No provisionamento preditivo, a decisão é tomada a partir de uma estimativa $\hat{d}_{t+h}$ da demanda futura.

A desvantagem da abordagem reativa é o atraso. Entre a detecção de um aumento de carga e a disponibilidade de uma nova instância há o tempo de inicialização, que não é desprezível e varia entre provedores e configurações (MAO; HUMPHREY, 2012). Durante esse intervalo, a aplicação opera com capacidade insuficiente. A abordagem preditiva tenta antecipar a demanda com um horizonte h pelo menos igual a esse tempo de inicialização, de modo que a instância já esteja disponível quando a carga chegar. Provedores comerciais já oferecem modos de escalonamento preditivo, como o do Amazon EC2 Auto Scaling (Amazon Web Services, 2024), o que reforça a relevância prática do problema.

Qualquer que seja a abordagem, cada decisão pode errar em duas direções. Há superprovisionamento quando a capacidade alocada excede a demanda ($k \cdot n_t > d_t$): o serviço é atendido, mas a capacidade ociosa $k \cdot n_t - d_t$ representa custo sem retorno. Há subprovisionamento quando a capacidade é menor que a demanda ($k \cdot n_t < d_t$): o custo é menor, mas parte da demanda não é atendida e o SLA pode ser violado. O objetivo da alocação é equilibrar essas duas falhas, e a qualidade da previsão determina o quanto esse equilíbrio pode ser alcançado.

2.1.4 Métricas de avaliação

As métricas usadas na literatura se dividem em dois grupos, e a distinção entre eles é central para este trabalho.

Métricas de previsão. Medem o quanto a previsão se aproxima do valor real, independentemente de como ela será usada. Para N previsões $\hat{y}_i$ e valores reais y_i, as mais comuns são:

$$\mathrm{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2, \qquad \mathrm{RMSE} = \sqrt{\mathrm{MSE}}, \tag{2-1}$$

$$\mathrm{MAE} = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i|, \qquad \mathrm{MAPE} = \frac{100\%}{N}\sum_{i=1}^{N}\left|\frac{y_i - \hat{y}_i}{y_i}\right|. \tag{2-2}$$

O MAPE é indefinido quando algum y_i é zero e instável quando y_i é próximo de zero, situação comum em períodos de baixa utilização. Além disso, as quatro métricas são simétricas: superestimar e subestimar a demanda pela mesma quantidade produz o mesmo erro. Para a alocação, porém, esses erros têm consequências diferentes, já que a superestimação gera custo e a subestimação gera violações. Por isso, um modelo com menor RMSE não é necessariamente o que leva às melhores decisões de alocação.

Métricas de alocação. Medem o resultado das decisões tomadas a partir da previsão. Para uma simulação com T intervalos, as métricas adotadas neste trabalho são:

$$C = \sum_{t=1}^{T} c \cdot n_t \qquad \text{(custo operacional total)} \tag{2-3}$$

$$V = \frac{1}{T}\sum_{t=1}^{T} \mathbf{1}\left[\, d_t > k \cdot n_t \,\right] \qquad \text{(fração de intervalos com violação)} \tag{2-4}$$

$$U = \frac{\sum_t \max(0,\, d_t - k \cdot n_t)}{\sum_t d_t} \qquad \text{(fração da demanda não atendida)} \tag{2-5}$$

$$O = \frac{\sum_t \max(0,\, k \cdot n_t - d_t)}{\sum_t k \cdot n_t} \qquad \text{(fração da capacidade ociosa)} \tag{2-6}$$

Também é útil contar o número de alterações em n_t ao longo da simulação, pois políticas que oscilam com frequência são indesejáveis na prática. Essas métricas correspondem diretamente aos termos da formulação da Equação 1-1: C é a função objetivo e V é o lado esquerdo da restrição.

As métricas de nível de serviço propostas por Rossi et al. (2025) ficam entre os dois grupos. A taxa de sucesso mede a fração de intervalos em que a demanda real fica abaixo do limite superior da previsão, e o total de recursos previstos soma esses limites superiores em relação à demanda real. Ambas são calculadas sobre a própria previsão, mas se aproximam, respectivamente, do complemento de V e do custo C no caso em que a capacidade pode ser ajustada de forma contínua. Este trabalho dá um passo adiante ao aplicar a previsão a uma política de alocação com instâncias discretas e medir as métricas de alocação diretamente.

2.2 Abordagens para previsão de carga

2.2.1 Modelos estatísticos

Os métodos mais simples de previsão servem como referência para todos os demais (HYNDMAN; ATHANASOPOULOS, 2018). O método ingênuo, ou de persistência, assume que o próximo valor será igual ao último observado, $\hat{y}_{t+1} = y_t$. A média móvel usa a média das w últimas observações. A suavização exponencial simples atribui pesos que decaem geometricamente com a idade da observação:

$$\hat{y}_{t+1} = \alpha\, y_t + (1-\alpha)\, \hat{y}_t, \qquad 0 < \alpha \leq 1, \tag{2-7}$$

em que valores maiores de α fazem o modelo reagir mais rapidamente a mudanças. O método de Holt-Winters estende essa ideia com componentes de tendência e de sazonalidade, o que o torna adequado a séries com padrões periódicos, como os ciclos diários comuns em cargas de nuvem (HYNDMAN; ATHANASOPOULOS, 2018).

A família ARIMA (*AutoRegressive Integrated Moving Average*), sistematizada pela metodologia de Box e Jenkins (BOX et al., 2015), é o modelo estatístico mais usado em previsão de séries temporais. Um modelo ARIMA(p, d, q) aplica d diferenciações para tornar a série estacionária e descreve o resultado como combinação linear de p valores passados (componente autorregressiva) e de q erros de previsão passados (componente de média móvel). Usando o operador de defasagem B, definido por $B\, y_t = y_{t-1}$, o modelo é escrito como:

$$\left(1 - \sum_{i=1}^{p} \phi_i B^i\right) (1-B)^d\, y_t = \beta_0 + \left(1 + \sum_{j=1}^{q} \theta_j B^j\right) a_t, \tag{2-8}$$

em que ϕ_i e θ_j são os coeficientes estimados, β_0 é uma constante e a_t é um ruído branco. A variante sazonal, SARIMA, acrescenta termos equivalentes defasados pelo período da sazonalidade. A escolha das ordens p, d e q pode ser feita pela análise das funções de autocorrelação ou por critérios de informação, como o AIC (HYNDMAN; ATHANASOPOULOS, 2018).

No contexto de nuvem, Calheiros et al. (2015) aplicaram um modelo ARIMA a rastros reais de requisições à Wikipédia em inglês, agregados por hora, para apoiar o provisionamento proativo de recursos de provedores de *software* como serviço. O modelo prevê a demanda da hora seguinte, e um módulo de desempenho converte essa previsão no número de máquinas virtuais necessárias. Além da precisão da previsão, que chegou a 91% em média, os autores avaliaram em simulação, com o CloudSim (CALHEIROS et al., 2011), o efeito das previsões sobre a utilização dos recursos e a qualidade de serviço. Um ponto especialmente relevante para este trabalho é que eles repetiram a simulação usando, em vez do valor previsto, os limites inferiores e superiores dos intervalos de confiança de 80% e 95%. Usar os limites superiores reduziu as requisições rejeitadas e as violações de QoS, ao custo de mais horas de máquina virtual, o que mostra de forma concreta o equilíbrio entre custo e violações controlado por um ajuste aplicado depois da previsão. A avaliação, contudo, envolve um único modelo de previsão.

Limitações. Os modelos estatísticos clássicos assumem relações lineares e, no caso do ARIMA, estacionariedade após a diferenciação. Cargas de nuvem frequentemente violam essas hipóteses, com mudanças abruptas de regime e picos que não seguem um padrão linear. Esses modelos também são essencialmente univariados em sua forma básica e precisam ser reajustados quando o comportamento da série muda. Por outro lado, eles exigem poucos dados, são baratos de ajustar e têm parâmetros interpretáveis. Por isso, devem estar presentes em qualquer comparação: uma rede neural só se justifica se superar essas alternativas mais simples sob as mesmas condições.

2.2.2 Redes neurais recorrentes

Para aplicar uma rede neural à previsão, a série é transformada em um problema de aprendizado supervisionado por meio de uma janela deslizante: cada exemplo tem como entrada as w observações mais recentes e como saída o valor a ser previsto. Para vários horizontes, o modelo pode ser aplicado de forma recursiva, realimentando as próprias previsões e acumulando erro, ou de forma direta, com uma saída para cada horizonte (TAIEB et al., 2012).

As redes neurais recorrentes (RNN) leem a janela em ordem e mantêm um estado interno que funciona como memória do que já foi visto, o que as torna adequadas a séries temporais. Em sequências longas, porém, o treinamento tem dificuldade de aprender relações entre observações distantes (BENGIO; SIMARD; FRASCONI, 1994). A LSTM resolve esse problema com portas que aprendem o que guardar e o que esquecer (HOCHREITER; SCHMIDHUBER, 1997), e a GRU simplifica essa ideia com apenas duas portas (CHO et al., 2014), sendo mais barata de treinar e com desempenho comparável (CHUNG et al., 2014).

O esDNN (XU et al., 2022) combina uma GRU modificada com camadas convolucionais para prever o uso de processador e de memória nos rastros da Alibaba e do Google. Nos experimentos, a GRU pura é competitiva em horizontes curtos, e o esDNN se destaca em horizontes longos. A avaliação, no entanto, apoia-se em métricas de erro simétricas, que não distinguem superestimação de subestimação (Seção 2.1.4).

Limitações. Redes recorrentes exigem mais dados e mais ajuste do que os modelos estatísticos, e, como qualquer previsor treinado para minimizar um erro médio, tendem a suavizar os picos de demanda. Agulló López et al. (2025) mostram que previsores clássicos e profundos falham justamente nesses picos, sem que as métricas usuais revelem a falha, e que ajustar a saída da previsão melhora a antecipação dos picos ao custo de mais superprovisionamento. Isso reforça que o erro médio não basta para avaliar um previsor usado na alocação.

2.2.3 Modelos com quantificação de incerteza

Rossi et al. (2025) propõem modelos bayesianos profundos (HBNN e LSTMD), em versões univariada e bivariada, que preveem não apenas um valor, mas uma distribuição para a demanda futura de processador e de memória. Os experimentos usam rastros do Google e da Alibaba e comparam os modelos com uma LSTM determinística. Os autores mostram que as métricas de erro pontual não distinguem bem os modelos, enquanto métricas de nível de serviço, calculadas sobre o limite superior do intervalo de confiança, evidenciam a vantagem de modelar a incerteza. O trabalho também investiga transferência de aprendizado e conclui que ela funciona razoavelmente dentro de um mesmo provedor, mas se degrada entre provedores diferentes.

Limitações. O efeito da incerteza é medido por métricas calculadas sobre a própria previsão, sem acoplar os modelos a um mecanismo de alocação, e os próprios autores apontam essa integração como trabalho futuro (ROSSI et al., 2025). Além disso, as métricas de nível de serviço supõem que a capacidade possa ser ajustada de forma contínua, enquanto, na prática, os recursos são alocados em instâncias discretas, com atraso de inicialização. Ainda assim, o trabalho sugere uma ideia simples e útil: provisionar pelo limite superior de um intervalo de previsão, e não pelo valor central, é uma forma de trocar custo por menos violações. A margem de segurança m apresentada na Seção 2.3.2 é a versão determinística dessa mesma ideia e é a adotada neste trabalho.

2.3 Abordagens para alocação de recursos

2.3.1 Heurísticas e políticas reativas

As políticas baseadas em limiares são a forma mais difundida de escalonamento automático, sobretudo pela simplicidade (LORIDO-BOTRAN; MIGUEL-ALONSO; LOZANO, 2014). Na forma mais comum, define-se um limiar superior θ_{sup} e um inferior θ_{inf} para a utilização média das instâncias: se a utilização permanece acima de θ_{sup} por um certo número de intervalos, uma instância é adicionada; se permanece abaixo de θ_{inf}, uma instância é removida. Um período de espera (*cooldown*) após cada ação impede que o sistema oscile.

Uma variação proporcional é usada pelo *Horizontal Pod Autoscaler* do Kubernetes, que calcula o número desejado de réplicas a partir da razão entre a métrica observada u_t e um valor-alvo u^* (The Kubernetes Authors, 2024):

$$n_{t+1} = \left\lceil n_t \cdot \frac{u_t}{u^*} \right\rceil . \tag{2-9}$$

O mecanismo inclui ainda uma faixa de tolerância, dentro da qual nenhuma ação é tomada, e uma janela de estabilização para reduções, ambas destinadas a evitar oscilações. Outro exemplo de política reativa é a usada como referência pelo

esDNN, que define o número de máquinas ativas como a média do número de máquinas dos cinco intervalos anteriores (XU et al., 2022).

Um problema relacionado, mas distinto, é o posicionamento das instâncias nos servidores físicos. Ele é tratado como uma variante do problema de empacotamento (*bin packing*), que é NP-difícil, e por isso costuma ser resolvido com heurísticas como *First-Fit* e *Best-Fit Decreasing*. Beloglazov e Buyya (2012), por exemplo, adaptam esta última para consolidar máquinas virtuais e reduzir o consumo de energia. Meta-heurísticas como algoritmos genéticos, otimização por enxame de partículas e colônia de formigas exploram o espaço de soluções de forma mais ampla, ao custo de maior tempo de computação (BODRA; KHAIRNAR, 2025). Como definido na Seção 1.4, o posicionamento está fora do escopo deste trabalho, que decide apenas quantas instâncias manter ativas.

Limitações. Por construção, as políticas reativas só respondem depois que a mudança de carga já ocorreu, e o tempo de inicialização das instâncias amplia esse atraso. Os limiares precisam ser ajustados manualmente para cada aplicação: limiares agressivos causam oscilações, e limiares conservadores mantêm capacidade ociosa. Ainda assim, essas políticas são simples, previsíveis e amplamente usadas, o que faz da política reativa por limiar uma referência obrigatória na avaliação deste trabalho.

2.3.2 Alocação guiada por previsão

Na alocação guiada por previsão, a decisão é tomada em duas etapas desacopladas. Primeiro, um modelo estima a demanda $\hat{d}_{t+h}$ para o intervalo em que a decisão terá efeito. Em seguida, uma regra converte essa estimativa em número de instâncias, normalmente dividindo a demanda prevista pela capacidade de uma instância e acrescentando uma margem de segurança $m \geq 0$:

$$n_{t+h} = \left\lceil \frac{(1+m)\,\hat{d}_{t+h}}{k} \right\rceil . \tag{2-10}$$

A margem controla o equilíbrio entre custo e violações: margens maiores reduzem V e aumentam C. Escolhê-la, calibrando em um conjunto de validação, é a forma prática de buscar o menor custo que respeita o limite ε da Equação 1-1. Quando o modelo fornece uma distribuição, a margem pode ser substituída por um quantil, como fazem Rossi et al. (2025) e Calheiros et al. (2015) ao provisionar pelo limite superior de um intervalo de confiança. Além disso, o horizonte h deve cobrir o tempo de inicialização das instâncias (MAO; HUMPHREY, 2012).

O MagicScaler (PAN et al., 2023) leva essa ideia mais longe: combina uma previsão com incerteza a uma otimização estocástica que minimiza o custo sujeito a um limite de demanda não atendida e, em *clusters* da Alibaba Cloud, reduziu substancialmente a perda de qualidade de serviço em relação a uma política reativa, com pequeno aumento de custo. Seus dados, porém, são privados, e o mecanismo de decisão é aplicado apenas ao próprio previsor, sem comparação

com outros modelos sob a mesma regra.

A principal vantagem do desacoplamento é a modularidade: com a regra de decisão fixa, é possível trocar apenas o modelo de previsão e atribuir a ele as diferenças de custo e de violações. É essa propriedade que este trabalho explora para comparar redes neurais e métodos tradicionais.

2.3.3 Aprendizado por reforço

A revisão de Bodra e Khairnar (2025) compara abordagens recentes de aprendizado de máquina para alocação de recursos em nuvem, incluindo métodos de aprendizado por reforço profundo, meta-heurísticas híbridas e modelos de previsão. Os autores observam que os trabalhos analisados são avaliados em ambientes, conjuntos de dados e métricas muito diferentes entre si, muitas vezes com cargas sintéticas, o que dificulta a comparação direta. Apontam ainda que os métodos de aprendizado por reforço profundo são sensíveis ao projeto da função de recompensa e aos hiperparâmetros e enfrentam limitações de escalabilidade conforme o espaço de estados cresce.

No aprendizado por reforço, a alocação é formulada como um problema de decisão sequencial. Um agente observa o estado do sistema, como a utilização recente e o número de instâncias ativas, escolhe uma ação, como adicionar, remover ou manter instâncias, e recebe uma recompensa que penaliza o custo e as violações de SLA. Ao longo de muitas interações, o agente aprende uma política que maximiza a recompensa acumulada. Diferentemente da alocação guiada por previsão, previsão e decisão não são separadas: a política mapeia diretamente estados em ações, e a antecipação da demanda fica implícita no que o agente aprende. O aprendizado por reforço aparece tanto na classificação de Lorido-Botran, Miguel-Alonso e Lozano (2014) quanto na revisão de Bodra e Khairnar (2025).

Limitações e posicionamento. Além da sensibilidade à função de recompensa e aos hiperparâmetros apontada por Bodra e Khairnar (2025), uma política aprendida é difícil de interpretar: quando ela toma uma decisão ruim, não é simples identificar se o problema está na antecipação da demanda ou na regra de decisão. O treinamento também exige um ambiente de simulação e um grande número de episódios. Por esses motivos, este trabalho adota a abordagem desacoplada da Seção 2.3.2, que permite isolar e medir o efeito da previsão sobre a alocação, e deixa o aprendizado por reforço fora do escopo (Seção 1.4). Comparar a abordagem desacoplada com uma política aprendida é uma extensão natural para trabalhos futuros.

2.4 Comparativo dos trabalhos relacionados

A Tabela 2.1 resume os trabalhos mais próximos desta proposta segundo os critérios relevantes para o problema definido na Seção 1.2: a técnica de previsão,

os dados usados, as métricas de avaliação, se a previsão é acoplada a uma decisão de alocação e se diferentes preditores são comparados sob a mesma política de alocação.

Tabela 2.1: Comparativo entre os trabalhos relacionados e a proposta deste trabalho.

Trabalho	**Técnica de previsão**	**Dados**	**Métricas**	**Acopla à alocação**	**Compara preditores sob a mesma política**
Calheiros et al. (2015)	ARIMA	Requisições à Wikipédia (1 h)	Precisão, rejeições, violações de QoS e horas de VM	Sim	Não
Xu et al. (2022)	GRU modificada com CNN, multivariada	Alibaba e Google	MSE, RMSE, MAPE e máquinas ativas	Parcial	Não
Pan et al. (2023)	Atenção multiescala com processo gaussiano	Alibaba Cloud (privados)	MSE, MAE, custo e perda de QoS	Sim	Não
Rossi et al. (2025)	Redes bayesianas (HBNN, LSTMD)	Alibaba e Google	Erro pontual e nível de serviço	Não	Não se aplica
Agulló López et al. (2025)	Clássicos, aprendizado de máquina e profundo, com saída ajustada	Alibaba e Google	Erro, detecção de picos e superprovisionamento	Não	Não se aplica
Bodra e Khairnar (2025)	Revisão: DRL, meta-heurísticas e previsores	Variados, muitos sintéticos	Heterogêneas entre os trabalhos	Não se aplica	Não se aplica
Este trabalho	GRU univariada com saída direta e cinco métodos clássicos	Alibaba e Google	Erro de previsão, custo, violações de SLA e ociosidade	Sim	Sim

A tabela evidencia uma separação entre duas linhas de trabalho. De um lado, os trabalhos de previsão (XU et al., 2022; ROSSI et al., 2025; AGULLÓ LÓPEZ et al., 2025) comparam vários modelos, mas avaliam-nos por métricas calculadas sobre a própria previsão. De outro, os trabalhos que acoplam a previsão à alocação (CALHEIROS et al., 2015; PAN et al., 2023) medem o efeito sobre o serviço, mas aplicam o mecanismo de decisão a um único preditor e o comparam com políticas reativas. Nenhum dos trabalhos analisados combina as duas coisas, isto é, submete vários preditores à mesma política de alocação e compara o custo e as violações resultantes.

2.4.1 Síntese e lacuna

A análise dos trabalhos relacionados leva a três observações. Primeiro, a maior parte da literatura de previsão avalia os modelos isoladamente, por métricas

de erro simétricas, que não refletem a assimetria entre superestimar a demanda, que gera custo, e subestimá-la, que gera violações (XU et al., 2022; ROSSI et al., 2025). Segundo, há evidências de que o erro médio é um guia incompleto para a alocação: previsores com bom desempenho nessas métricas falham justamente nos picos de demanda, que são os momentos em que as violações acontecem, e ajustes simples na saída da previsão mudam o equilíbrio entre custo e violações sem alterar o modelo (AGULLÓ LÓPEZ et al., 2025). Terceiro, quando a previsão é acoplada à alocação, a comparação costuma ser feita contra políticas reativas, com dados privados ou em ambientes e métricas heterogêneos (XU et al., 2022; PAN et al., 2023; BODRA; KHAIRNAR, 2025), o que impede atribuir as diferenças observadas ao preditor.

A lacuna que este trabalho busca preencher é, portanto, a de uma avaliação de ponta a ponta, com dados públicos de um *cluster* real, que meça o custo e as violações de SLA obtidos por preditores de complexidade crescente, do método ingênuo à rede neural, todos submetidos à mesma política de alocação e calibrados para o mesmo limite de violações ε. Desse modo, a pergunta deixa de ser qual modelo erra menos e passa a ser quanto custo cada modelo economiza para um mesmo nível de serviço. O Capítulo 3 descreve a solução proposta para essa avaliação e os objetivos específicos do trabalho.

2.5 Tecnologias utilizadas

Esta seção descreve as linguagens, bibliotecas e ferramentas adotadas, organizadas conforme as etapas do *pipeline*: tratamento dos dados, previsão de carga, simulação da alocação e ambiente de execução. Todo o código é escrito em Python 3.12, linguagem escolhida pela disponibilidade de bibliotecas maduras para séries temporais e aprendizado profundo, e está disponível em repositório público (ROSAS, 2026).

2.5.1 Tratamento dos dados

A tabela de uso de recursos por máquina do rastro da Alibaba de 2018 (Alibaba Group, 2018) ocupa cerca de 1,7 GB compactada. Para agregá-la em uma série de demanda com intervalos de cinco minutos sem carregar o arquivo inteiro na memória, é usado o DuckDB (RAASVELDT; MÜHLEISEN, 2019), um banco de dados analítico embutido que executa consultas SQL diretamente sobre arquivos CSV. As séries resultantes são gravadas em formato Parquet e manipuladas com *pandas* (MCKINNEY, 2010) e NumPy (HARRIS et al., 2020). A mesma série alimenta todos os modelos, o que garante a comparabilidade entre eles.

2.5.2 Modelos de previsão

Os métodos ingênuo, de média móvel e sazonal ingênuo são implementados diretamente com NumPy. O ARIMA e o Holt-Winters usam a biblioteca *statsmodels* (SEABOLD; PERKTOLD, 2010) para a estimação dos parâmetros no conjunto de treino. A rede neural é implementada e treinada com PyTorch (PASZKE et al., 2019), que oferece flexibilidade para definir arquiteturas recorrentes e personalizar o ciclo de treinamento. Como a rede é pequena, é usada a versão do PyTorch apenas para CPU, o que dispensa GPU para reproduzir os experimentos.

2.5.3 Simulação

A proposta inicial previa o uso do simulador CloudSim Plus (FILHO et al., 2017), um *framework* em Java derivado do CloudSim (CALHEIROS et al., 2011). O CloudSim é um simulador de eventos discretos que modela centros de dados, servidores, máquinas virtuais e a execução de tarefas, incluindo políticas de compartilhamento dos núcleos de processamento entre máquinas virtuais e entre tarefas (CALHEIROS et al., 2011), e foi usado, por exemplo, para avaliar o provisionamento guiado por ARIMA de Calheiros et al. (2015). Durante o Projeto Final I, optou-se por implementar um simulador próprio, em Python, por três motivos. Primeiro, o CloudSim Plus modela em detalhe a execução de tarefas e o posicionamento de máquinas virtuais em servidores físicos, que estão fora do escopo deste trabalho (Seção 1.4), enquanto a decisão estudada aqui se resume a escolher o número de instâncias n_t a cada intervalo, a partir da demanda agregada. Segundo, manter previsão e simulação na mesma linguagem evita uma ponte entre Python e Java para transferir as previsões. Terceiro, um simulador pequeno, em tempo discreto, pode ser coberto por testes automatizados e conferido intervalo a intervalo, o que é necessário para garantir que as diferenças entre preditores não sejam artefatos da simulação. O simulador implementa o atraso de provisionamento, as políticas descritas neste capítulo e as métricas da Seção 2.1.4. As figuras são geradas com Matplotlib (HUNTER, 2007).

2.5.4 Ambiente de execução e reprodutibilidade

As dependências são gerenciadas com o *uv*, que fixa as versões de todas as bibliotecas em um arquivo de travamento, e o projeto pode ser executado também em um contêiner Docker. Um *Makefile* encadeia as etapas do *pipeline*, de modo que todas as tabelas e figuras podem ser regeneradas a partir dos dados brutos com um único comando. Todos os parâmetros dos experimentos, como a divisão dos dados, os horizontes de previsão e as grades de margens, ficam em um arquivo de configuração, e nenhum é fixado no código. A correção

dos componentes é verificada com testes automatizados escritos com *pytest*, e o código é versionado com Git no GitHub.

3
Objetivos do trabalho

4
Plano de ação

5
Cronogramas

Referências

AGULLÓ LÓPEZ, F. et al. Enhancing the output of time series forecasting algorithms for cloud resource provisioning. **Future Generation Computer Systems**, v. 170, p. 107833, 2025.

Alibaba Group. **Alibaba Cluster Trace Program**. 2018. Disponível em: https://github.com/alibaba/clusterdata.

Amazon Web Services. **Predictive scaling for Amazon EC2 Auto Scaling**. 2024. Documentação do Amazon EC2 Auto Scaling. Disponível em: https://docs.aws.amazon.com/autoscaling/ec2/userguide/ec2-auto-scaling-predictive-scaling.html. Acesso em: 14 out. 2026.

BELOGLAZOV, A.; BUYYA, R. Optimal online deterministic algorithms and adaptive heuristics for energy and performance efficient dynamic consolidation of virtual machines in cloud data centers. **Concurrency and Computation: Practice and Experience**, v. 24, n. 13, p. 1397–1420, 2012.

BENGIO, Y.; SIMARD, P.; FRASCONI, P. Learning long-term dependencies with gradient descent is difficult. **IEEE Transactions on Neural Networks**, v. 5, n. 2, p. 157–166, 1994.

BEYER, B. et al. **Site Reliability Engineering: How Google Runs Production Systems**. Sebastopol, CA: O'Reilly Media, 2016.

BODRA, D.; KHAIRNAR, S. Machine learning-based cloud resource allocation algorithms: a comprehensive comparative review. **Frontiers in Computer Science**, v. 7, p. 1678976, 2025.

BOX, G. E. P. et al. **Time Series Analysis: Forecasting and Control**. 5. ed. Hoboken, NJ: John Wiley & Sons, 2015.

CALHEIROS, R. N. et al. Workload prediction using ARIMA model and its impact on cloud applications' QoS. **IEEE Transactions on Cloud Computing**, v. 3, n. 4, p. 449–458, 2015.

CALHEIROS, R. N. et al. CloudSim: a toolkit for modeling and simulation of cloud computing environments and evaluation of resource provisioning algorithms. **Software: Practice and Experience**, v. 41, n. 1, p. 23–50, 2011.

CHO, K. et al. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In: **Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)**. [S.l.: s.n.], 2014. p. 1724–1734.

CHUNG, J. et al. Empirical evaluation of gated recurrent neural networks on sequence modeling. In: **NIPS 2014 Workshop on Deep Learning**. [S.l.: s.n.], 2014. Disponível em: https://arxiv.org/abs/1412.3555.

FILHO, M. C. S. et al. CloudSim Plus: a cloud computing simulation framework pursuing software engineering principles for improved modularity, extensibility and correctness. In: **2017 IFIP/IEEE Symposium on Integrated Network and Service Management (IM)**. [S.l.: s.n.], 2017. p. 400–406.

HARRIS, C. R. et al. Array programming with NumPy. **Nature**, v. 585, p. 357–362, 2020.

HERBST, N. R.; KOUNEV, S.; REUSSNER, R. Elasticity in cloud computing: What it is, and what it is not. In: **Proceedings of the 10th International Conference on Autonomic Computing (ICAC 13)**. San Jose, CA: USENIX, 2013. p. 23–27.

HOCHREITER, S.; SCHMIDHUBER, J. Long short-term memory. **Neural Computation**, v. 9, n. 8, p. 1735–1780, 1997.

HUNTER, J. D. Matplotlib: A 2D graphics environment. **Computing in Science & Engineering**, v. 9, n. 3, p. 90–95, 2007.

HYNDMAN, R. J.; ATHANASOPOULOS, G. **Forecasting: principles and practice**. 2. ed. Melbourne: OTexts, 2018. Disponível em: https://otexts.com/fpp2/.

LORIDO-BOTRAN, T.; MIGUEL-ALONSO, J.; LOZANO, J. A. A review of auto-scaling techniques for elastic applications in cloud environments. **Journal of Grid Computing**, v. 12, n. 4, p. 559–592, 2014.

MAO, M.; HUMPHREY, M. A performance study on the VM startup time in the cloud. In: **2012 IEEE Fifth International Conference on Cloud Computing (CLOUD)**. [S.l.: s.n.], 2012. p. 423–430.

MCKINNEY, W. Data structures for statistical computing in Python. In: WALT, S. van der; MILLMAN, J. (Ed.). **Proceedings of the 9th Python in Science Conference**. [S.l.: s.n.], 2010. p. 56–61.

MELL, P.; GRANCE, T. **The NIST Definition of Cloud Computing**. Gaithersburg, MD, 2011.

PAN, Z. et al. MagicScaler: Uncertainty-aware, predictive autoscaling. **Proceedings of the VLDB Endowment**, v. 16, n. 12, p. 3808–3821, 2023.

PASZKE, A. et al. PyTorch: An imperative style, high-performance deep learning library. In: **Advances in Neural Information Processing Systems 32 (NeurIPS 2019)**. [S.l.: s.n.], 2019.

RAASVELDT, M.; MÜHLEISEN, H. DuckDB: an embeddable analytical database. In: **Proceedings of the 2019 International Conference on Management of Data (SIGMOD '19)**. [S.l.]: ACM, 2019. p. 1981–1984.

ROSAS, G. L. d. S. **predictive-cloud-autoscaling: código do Projeto Final**. 2026. Disponível em: https://github.com/gabriellsar/predictive-cloud-autoscaling. Acesso em: 20 out. 2026.

ROSSI, A. et al. Forecasting workload in cloud computing: towards uncertainty-aware predictions and transfer learning. **Cluster Computing**, v. 28, n. 4, p. 258, 2025. Código e dados disponíveis em: https://github.com/andreareds/TowardsUncertaintyAwareWorkloadPrediction.

SEABOLD, S.; PERKTOLD, J. statsmodels: Econometric and statistical modeling with Python. In: **Proceedings of the 9th Python in Science Conference**. [S.l.: s.n.], 2010.

TAIEB, S. B. et al. A review and comparison of strategies for multi-step ahead time series forecasting based on the NN5 forecasting competition. **Expert Systems with Applications**, v. 39, n. 8, p. 7067–7083, 2012.

The Kubernetes Authors. **Horizontal Pod Autoscaling**. 2024. Documentação do Kubernetes. Disponível em: https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/. Acesso em: 14 out. 2026.

WILKES, J. **Google cluster-usage traces v3**. Mountain View, CA, USA, 2020. Disponível em: https://github.com/google/cluster-data.

XU, M. et al. esDNN: Deep neural network based multivariate workload prediction in cloud computing environments. **ACM Transactions on Internet Technology**, v. 22, n. 3, p. 75:1–75:24, 2022.